The Program

< Back to *The Input Files* | Forward to *Program Output* >

This page contains an overview of the program (**read.c**) that was used to test text file processing. The source code and the sample text files are also available:

- View the source code in your browser.
- Download the source code and text files.

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >

Program Objective

The program attempts to read a text file line-by-line and print out exactly what it read for each line. The output is in the form of a modified "hex dump" that may be compared directly to the actual contents of the input files. *All* characters read are retained in the output, including any line terminators and other special characters. The program doesn't "do" anything with the data in the text files; it merely displays what it read.

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >

Syntax

The syntax for running the program is:

```
read FileName OpenMode ReadFunction

where: FileName     = Text file to read

       OpenMode     = "t" for text mode, or
                      "b" for binary mode

       ReadFunction = "fscanf", or
                      "fgets",  or
                      "fgetc",  or
                      "fread"
```

All arguments are required. Don't include the quotes. *OpenMode* and *ReadFunction* must be in lowercase. All output is written to **stdout**, so you may redirect the output to a file.

The C read functions used are contained in separate functions within the program, and each is implemented differently. The remainder of this page briefly describes each function.

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >

Reading with *fscanf*

The program attempts to read a line from the text file with the following function call:

```
fscanf(input, "%s", output);
```

This will work provided that the text file *does not contain any space characters*. Since the sample text files *do* contain a space in one of the text strings, fscanf() *cannot* read the files properly.

As I indicated previously, fscanf() is a poor choice for reading most text files. No output is included here for this function.

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >

Reading with *fgets*

The program attempts to read a line from the text file with the following function call:

```
fgets(output, MAX_REC_LEN, input);
```

"MAX_REC_LEN" is a symbolic constant defined in the program as:

```
#define MAX_REC_LEN 1024
```

This allows fgets() to read records of up to one kilobyte in length (actually, 1K - 1, since one character is reserved for the null-terminator). This is more than adequate for the sample files, but a "real" application should account for the possibility of larger records (and they *do* occur).

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >

Reading with *fgetc*

Since fgetc() only reads one character at a time, we can't return an entire line using only one function call. To read an entire line using fgetc(), the program repeatedly makes the following function call:

```
iThisChar = fgetc(input);
```

and accumulates the output in a character array.

It continues reading one character at a time until it encounters the first character *following* a CR or LF that is *not* a CR or LF (or reaches the end of file). It assumes that everything up to (but *not* including) this character is part of the current line.

As a result, the function will *not* read **blank** lines correctly. It will include *all* successive CRs and LFs as the trailing characters on the current line. A "real" application should take this into account.

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >

Reading with *fread*

In this program, the logic behind reading a line with fread() is similar to that of fgetc(), except that instead of reading one character at a time from disk, it makes the *single* function call:

```
fread(cFile, lFileLen, 1, input);
```

to read the *entire file* into the array **cFile**. It still parses one character at a time, but it does it from the array rather than from disk, which is significantly faster.

The program *dynamically allocates* the size of the **cFile** array, so we don't need to know the size of the file in advance. Of course, it's possible that there may be insufficient memory to read in an entire file. See the source code for more information.

< Back to *The Input Files* | ^ Up to Top | Forward to *Program Output* >
